

# TaskApp Academica

Paso a paso del desarrollo de una aplicacion movil hibrida para gestionar tareas academicas con Ionic, Angular, dashboard visual, filtros y persistencia local.

## 1. Que pide la practica

La practica pide construir una aplicacion movil hibrida donde un estudiante pueda ver, crear, marcar como completadas y eliminar tareas academicas. El desarrollo se realizo con Ionic y Angular, aplicando componentes moviles, formularios reactivos, servicio Angular y persistencia local.

- Crear una pagina principal con componentes Ionic: ion-header, ion-title, ion-content e ion-footer.
- Mostrar tareas en una lista movil usando ion-list, ion-item e ion-label.
- Mostrar titulo, descripcion corta y registrar prioridad en el formulario.
- Usar un boton flotante ion-fab para abrir el formulario de nueva tarea.
- Definir una interfaz TypeScript llamada Tarea.
- Crear el formulario con ReactiveFormsModule y validaciones.
- Permitir marcar tareas como completadas con ion-checkbox y texto tachado.
- Permitir eliminar tareas con icono de papelera o deslizamiento.
- Como extra, centralizar la logica en un servicio Angular y guardar en localStorage.
- Mejorar la experiencia visual con resumen, barra de progreso y filtros por estado.

## 2. Estructura del proyecto

El proyecto se organizo con una estructura clara para separar configuracion, modelo de datos, servicio y pagina principal.

| Archivo o carpeta                            | Funcion                                                                            |
|----------------------------------------------|------------------------------------------------------------------------------------|
| <b>package.json</b>                          | Define dependencias y comandos como npm install, npm run start y npm run build.    |
| <b>angular.json</b>                          | Configura el proyecto Angular usado por Ionic.                                     |
| <b>ionic.config.json</b>                     | Indica a Ionic que el proyecto es de tipo Angular.                                 |
| <b>src/app/models/tarea.model.ts</b>         | Define la interfaz Tarea y el tipo de prioridad.                                   |
| <b>src/app/services/tareas.service.ts</b>    | Centraliza crear, eliminar, completar y guardar tareas en localStorage.            |
| <b>src/app/pages/tareas/tareas.page.ts</b>   | Contiene la logica de pantalla, signals, filtros y navegacion al registro.         |
| <b>src/app/pages/tareas/tareas.page.html</b> | Contiene dashboard, metricas, filtros, lista, checkbox, contador y boton flotante. |
| <b>src/app/pages/tareas/tareas.page.scss</b> | Define el diseno movil limpio, tarjetas, colores, estados y responsive.            |
| <b>src/app/pages/nueva-tarea/</b>            | Contiene la pagina independiente con formulario reactivo para registrar tareas.    |
| <b>README.md</b>                             | Explica los comandos para instalar y ejecutar el proyecto.                         |

## 3. Modelo de datos

Se definio una interfaz TypeScript para asegurar que todas las tareas tengan la misma forma. Esto ayuda a evitar errores y cumple el requisito del modelo de datos.

```
export interface Tarea {  
  id: number;  
  titulo: string;  
  descripcion: string;  
  prioridad: 'Alta' | 'Media' | 'Baja';  
  completada: boolean;  
}
```

## 4. Servicio de tareas

El servicio TareasService es el lugar donde se maneja el arreglo de tareas. Tiene metodos para agregar, cambiar estado y eliminar. Ademas, guarda los cambios en localStorage.

- agregarTarea crea una nueva tarea con id, titulo, descripcion, prioridad y completada en falso.
- cambiarEstado actualiza si una tarea esta completada o pendiente.
- eliminarTarea quita una tarea de la lista.
- actualizarTareas guarda el arreglo en localStorage.
- cargarTareas lee localStorage y carga datos iniciales si no hay informacion guardada.

## 5. Interfaz principal

La pantalla principal se diseña como una app móvil académica: cabecera limpia, hero con resumen del día, barra de progreso, tarjetas de métricas, filtros por estado, lista de tareas y acceso a una página de registro.

- ion-header e ion-title: parte superior de la app.
- ion-content: área principal donde vive el contenido.
- ion-progress-bar: muestra avance de tareas completadas.
- Botones de filtro: permiten ver todas, pendientes y completadas.
- ion-list e ion-item: lista móvil de tareas.
- ion-checkbox: cambia el estado completada.
- ion-badge: muestra el contador de tareas filtradas.
- ion-button con ion-icon trash: elimina una tarea.
- ion-item-sliding: permite eliminar deslizando el elemento.
- ion-fab: botón flotante para abrir el formulario.
- ion-footer: pie de la pantalla.

## 6. Formulario reactivo

El formulario está en una página independiente llamada nueva-tarea. Se usa `ReactiveFormsModule` para controlar campos y validaciones desde TypeScript.

- título: obligatorio y mínimo 5 caracteres.
- descripción: obligatoria y limitada a 120 caracteres.
- prioridad: se guarda por defecto como Media para mantener el formulario simple.
- Si el formulario es inválido, se muestran mensajes de error y no se guarda la tarea.

## 7. Paso a paso de desarrollo

- 1 Crear la carpeta del proyecto y abrirla en Visual Studio Code.
- 2 Crear la configuración base de Angular e Ionic: `package.json`, `angular.json` e `ionic.config.json`.
- 3 Instalar dependencias con `npm install`.
- 4 Crear el modelo Tarea en `src/app/models/tarea.model.ts`.
- 5 Crear el servicio `TareasService` para manejar los datos y `localStorage`.
- 6 Crear la página `TareasPage` con componentes standalone de Ionic.
- 7 Construir la plantilla HTML con dashboard, métricas, filtros, lista, página de registro, fab y footer.
- 8 Agregar estilos SCSS para una presentación limpia, compacta y móvil.
- 9 Probar la compilación con `npm run build`.

10 Ejecutar en desarrollo con ionic serve o npm run start.

## 8. Comandos necesarios

Instalar dependencias:

```
npm install
```

Ejecutar en desarrollo:

```
npm run start
```

Ejecutar el frontend con nombre explicito:

```
npm run start:frontend
```

Tambien puede ejecutarse directamente si se usa Ionic CLI:

```
ionic serve
```

Generar build de produccion:

```
npm run build
```

Generar build del frontend con nombre explicito:

```
npm run build:frontend
```

## 9. Entrega recomendada

- Subir el proyecto a GitHub sin la carpeta node\_modules.
- Verificar que .gitignore incluya node\_modules, www, dist y .angular.
- Adjuntar README.md con instrucciones de instalacion y ejecucion.
- Adjuntar este PDF como paso a paso del desarrollo.

## 10. Checklist final

- La app tiene interfaz Ionic completa.
- La app tiene dashboard, metricas, filtros y barra de progreso.
- La app permite crear tareas.
- La app valida el titulo obligatorio y minimo de 5 caracteres.
- La app permite marcar tareas como completadas.
- La app tacha el texto de tareas completadas.
- La app permite eliminar tareas.
- La app guarda datos en localStorage.
- El proyecto compila con npm run build.

Conclusion: el proyecto cumple los requerimientos de la practica mediante una aplicacion movil hibrida desarrollada con Ionic y Angular, organizada por componentes, servicio, modelo de datos, formulario reactivo y persistencia local.